



Task 1: Data Collection & Cleaning

Introduction

Raw data collected from different sources is often incomplete, inconsistent, and contains errors. This task focuses on collecting data and transforming it into a clean, structured format suitable for analysis and machine learning applications.

What I Did

- ✓ Collected data from external sources (CSV/Excel/API)
- ✓ Inspected dataset structure and data types
- ✓ Identified and handled missing values
- ✓ Removed duplicate records
- ✓ Corrected inconsistent data entries
- ✓ Prepared the dataset for further analysis

Technologies Used

- Python
- Pandas
- NumPy
- Jupyter Notebook

Data Cleaning Techniques

- Missing Value Treatment
- Duplicate Removal
- Data Validation
- Data Formatting
- Feature Standardization

Sample Operations

Remove missing values

`df.dropna()`

Fill missing values

`df.fillna()`

Remove duplicate records

`df.drop_duplicates()`

Dataset information

`df.info()`

Learning Outcomes

Through this task, I gained practical experience in:

- Data preprocessing
- Data quality assessment
- Data manipulation using Pandas
- Preparing datasets for analytics and machine learning workflows

```
import pandas as pd
```

```
import numpy as np
```

```
# Load data
```

```
# The previous error 'FileNotFoundError: [Errno 2] No  
such file or directory: 'data.csv''
```

```
# occurred because the file 'data.csv' was not found  
in the current environment.
```

```
# To fix this, you can either upload your 'data.csv' file  
or use an existing sample dataset.
```

```
# I've updated it to load  
'sample_data/california_housing_train.csv' as an  
example.
```

```
# If you have your own 'data.csv', please upload it  
and change the path accordingly.
```

```
df =  
pd.read_csv("sample_data/california_housing_train.  
csv")    # or pd.read_excel("data.xlsx")
```

```
# Check missing values
```

```
print(df.isnull().sum())
```

```
# Remove rows with missing values
```

```
df = df.dropna()
```

```
# OR fill missing values
```

```
# df = df.fillna(0)
```

```
# df = df.fillna(df.mean(numeric_only=True))
```

```
# Remove duplicate rows
```

```
df = df.drop_duplicates()
```

```
# Normalize a numeric column (example)
```

```
# Note: This part might need adjustment based on  
the actual columns in the loaded dataset.
```

```
# For 'california_housing_train.csv', 'median_income'
is a suitable numeric column.
```

```
if 'median_income' in df.columns:
```

```
    df['median_income_normalized'] = (
        (df['median_income'] -
         df['median_income'].min()) /
        (df['median_income'].max() -
         df['median_income'].min())
    )
```

```
# Display cleaned data
```

```
print(df.head())
```

```
# Save cleaned data
```

```
df.to_csv("cleaned_data.csv", index=False)
```

OUTPUT:

```

longitude      0
latitude       0
housing_median_age  0
total_rooms    0
total_bedrooms 0
population     0
households     0
median_income  0
median_house_value 0
dtype: int64

longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0   -114.31    34.19           15.0         5612.0         1283.0
1   -114.47    34.40           19.0         7650.0         1901.0
2   -114.56    33.69           17.0          720.0          174.0
3   -114.57    33.64           14.0        1501.0           337.0
4   -114.57    33.57           20.0        1454.0           326.0

population  households  median_income  median_house_value  \
0      1015.0        472.0         1.4936         66900.0
1      1129.0        463.0         1.8200         80100.0
2       333.0        117.0         1.6509         85700.0
3       515.0        226.0         3.1917         73400.0
4       624.0        262.0         1.9250         65500.0

median_income_normalized
0      0.068530
1      0.091040
2      0.079378
3      0.185639
4      0.098281

```

Conclusion

Data cleaning is a critical step in the data science lifecycle. A well-cleaned dataset improves the reliability, accuracy, and effectiveness of analysis and predictive models.